

Python Programming

Gap Topics — Self-Learning Guide

Advanced Python Concepts — Self-Learning Reference

What is this document? This guide covers topics that are either missing or were covered too lightly in the earlier Python Revision Course. Use this alongside that course — together they give you complete coverage of all Python concepts.

Topic	Contents
Topic 1	Lambda Functions, filter(), map(), reduce()
Topic 2	Advanced Function Arguments (*args, **kwargs, Docstrings)
Topic 3	String Methods: title(), capitalize(), swapcase(), maketrans(), translate(), startswith(), endswith(), % formatting, finditer()
Topic 4	Tuples: Packing, Unpacking, Singleton Rule, Copy, Comparison
Topic 5	Sets: frozenset, Set Comprehension, all(), any()
Topic 6	Dictionary Comprehension, Merging, Advanced Iteration
Topic 7	Modules: dir(), Namespaces, fromimport, __name__
Topic 8	Variable Scope & Lifetime, global, nonlocal, Closures
Ref Card	Complete Quick Reference — all gap topics on one page

Topic 1 — Lambda Functions, Filter, Map & Reduce

Functions — Anonymous functions and functional programming tools

What is a Lambda Function?

A **lambda** is a small, anonymous function defined in a single line. It can take any number of arguments but can only have **one expression**. It is mostly used where a function is needed temporarily — passed as an argument to another function.

Syntax: `lambda arguments : expression` Compare: `def square(x): return x**2` vs `lambda x: x**2`

```
# Basic lambda
square = lambda x: x**2
print(square(5)) # 25

# Lambda with multiple args
add = lambda a, b: a + b
print(add(3, 4)) # 7

# Lambda with condition (ternary)
classify = lambda n: "even" if n % 2 == 0 else "odd"
print(classify(7)) # odd

# Lambda used directly (no variable)
print((lambda x, y: x * y)(4, 5)) # 20
```

filter() — Keep elements that match a condition

filter(function, iterable) applies the function to each element and keeps only those for which the function returns True. Returns a filter object — wrap with `list()`.

```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Keep only even numbers
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens) # [2, 4, 6, 8, 10]

# Keep numbers greater than 5
big = list(filter(lambda x: x > 5, nums))
print(big) # [6, 7, 8, 9, 10]

# Filter out empty strings
words = ["hello", "", "world", "", "python"]
non_empty = list(filter(None, words)) # None keeps truthy values
print(non_empty) # ['hello', 'world', 'python']
```

map() — Transform every element

map(function, iterable) applies the function to every element and returns the transformed values. Can work on multiple iterables simultaneously.

```
nums = [1, 2, 3, 4, 5]

# Square every element
squares = list(map(lambda x: x**2, nums))
print(squares) # [1, 4, 9, 16, 25]

# Convert list of strings to integers
str_nums = ["10", "20", "30"]
ints = list(map(int, str_nums))
print(ints) # [10, 20, 30]

# map with TWO lists (pairs elements)
a = [1, 2, 3]
b = [10, 20, 30]
result = list(map(lambda x, y: x + y, a, b))
print(result) # [11, 22, 33]
```

reduce() — Accumulate to a single value

reduce(function, iterable) from the `functools` module applies the function cumulatively — left to right — until a single value remains. Think of it as folding the list into one result.

```
from functools import reduce

nums = [1, 2, 3, 4, 5]

# Product: 1*2*3*4*5 = 120
product = reduce(lambda a, b: a * b, nums)
print(product) # 120

# Sum (same as sum() built-in)
total = reduce(lambda a, b: a + b, nums)
print(total) # 15

# Find maximum
maximum = reduce(lambda a, b: a if a > b else b, nums)
print(maximum) # 5

# How reduce works step by step for [1,2,3,4,5]:
# Step 1: lambda(1,2) = 2
# Step 2: lambda(2,3) = 6
# Step 3: lambda(6,4) = 24
# Step 4: lambda(24,5) = 120
```

Tip: Use `reduce()` for cumulative operations (product, max, string join). For simple sum use the built-in `sum()` — it's faster.

Comparison: filter vs map vs reduce

Function	Purpose
<code>filter(fn, lst)</code>	Selects elements — returns subset of original list
<code>map(fn, lst)</code>	Transforms elements — returns same size, new values
<code>reduce(fn, lst)</code>	Combines elements — returns a SINGLE value

Combining all three — real example

```
from functools import reduce

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Step 1: filter — keep evens: [2,4,6,8,10]
# Step 2: map — square them: [4,16,36,64,100]
# Step 3: reduce — sum them: 220

result = reduce(
    lambda a, b: a + b,
    map(lambda x: x**2,
        filter(lambda x: x % 2 == 0, nums))
)

print(result) # 220
```

Remember: Lambda + filter/map/reduce is heavily tested. Know the difference: filter returns a subset, map returns a transformation, reduce returns ONE value.

Topic 2 — Advanced Function Arguments

Functions — `*args`, `**kwargs`, Default, Keyword, Variable-length arguments & Docstrings

All 5 argument types at a glance

Argument Type	Syntax example
Positional	<code>def f(a, b):</code> — order matters, required
Default	<code>def f(a, b=10):</code> — b is optional, has fallback
Keyword	<code>f(b=5, a=2)</code> — named at call time
<code>*args</code> (var positional)	<code>def f(*args):</code> — collects extra into tuple
<code>**kwargs</code> (var keyword)	<code>def f(**kwargs):</code> — collects extras into dict

`*args` — Variable Positional Arguments

Use `*args` when you don't know how many positional arguments will be passed. Inside the function, `args` is a **tuple**.

```
def total(*args):
    print(type(args)) #
    return sum(args)

print(total(1, 2, 3)) # 6
print(total(10, 20, 30, 40)) # 100
print(total()) # 0 (empty tuple)

# Mix with regular args — positional BEFORE *args
def greet(greeting, *names):
    for name in names:
        print(f"{greeting}, {name}!")

greet("Hello", "Raj", "Priya", "Sam")

# Hello, Raj!
# Hello, Priya!
# Hello, Sam!
```

`**kwargs` — Variable Keyword Arguments

Use `**kwargs` when you want to accept any number of named arguments. Inside the function, `kwargs` is a **dictionary**.

```
def display_info(**kwargs):
    print(type(kwargs)) #
    for key, value in kwargs.items():
        print(f"{key}: {value}")
```

```
display_info(name="Raj", age=20, city="Pune")

# name: Raj
# age: 20
# city: Pune

# Mix: regular + *args + **kwargs
def mixed(a, b, *args, **kwargs):
    print("a:", a)
    print("b:", b)
    print("extra args:", args)
    print("extra kwargs:", kwargs)

mixed(1, 2, 3, 4, x=10, y=20)

# a: 1
# b: 2
# extra args: (3, 4)
# extra kwargs: {'x': 10, 'y': 20}
```

Watch out: Order in definition MUST be: positional args, *args, keyword-only args, **kwargs. Wrong order = `SyntaxError`.

Documentation Strings (Docstrings)

A **docstring** is a string literal as the very first statement inside a function, class, or module. It documents what the function does. Access it via `func.__doc__`.

```
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The area (length * width).
    """
    return length * width

# Access the docstring
print(calculate_area.__doc__)

# help() also shows it
help(calculate_area)
```

Remember: Docstrings are NOT comments. Comments use `#` and are ignored by Python. Docstrings are stored as the `__doc__` attribute and can be accessed at runtime.

Topic 3 — String Methods Not Covered Earlier

Strings — `title()`, `capitalize()`, `swapcase()`, `maketrans()`, `translate()`, `startswith()`, `endswith()`, `finditer()`

Case Conversion Methods

Method	What it does
<code>s.upper()</code>	ALL CAPS
<code>s.lower()</code>	all lowercase
<code>s.title()</code>	First Letter Of Every Word Capitalized
<code>s.capitalize()</code>	First letter of ENTIRE string capitalized, rest lowercase
<code>s.swapcase()</code>	Swaps — uppercase becomes lower, lowercase becomes upper

```
s = "hello WORLD"
print(s.title()) # Hello World
print(s.capitalize()) # Hello world (only first char)
print(s.swapcase()) # HELLO world

# title() capitalizes EVERY word
s2 = "the quick brown fox"
print(s2.title()) # The Quick Brown Fox

# capitalize() only the first character of the whole string
print(s2.capitalize()) # The quick brown fox
```

Watch out: `capitalize()` and `title()` are commonly confused. `capitalize()` only does the first character of the entire string; `title()` does every word.

`startswith()` and `endswith()`

Both return `True` or `False`. They can accept a **tuple of prefixes/suffixes** to check multiple at once.

```
filename = "report_2024.pdf"

# Basic check
print(filename.startswith("report")) # True
print(filename.endswith(".pdf")) # True
print(filename.endswith(".docx")) # False

# Check multiple suffixes using a TUPLE
print(filename.endswith((".pdf", ".docx", ".xlsx"))) # True

# With start and end positions
s = "Hello World"
print(s.startswith("World", 6)) # True (start search from index 6)
```

```
print(s.endswith("Hello", 0, 5)) # True (search between index 0-5)

# Practical use: filter files
files = ["notes.txt", "data.csv", "image.png", "report.pdf"]
docs = [f for f in files if f.endswith((".txt", ".pdf"))]
print(docs) # ['notes.txt', 'report.pdf']
```

maketrans() and translate()

`maketrans()` creates a translation table mapping characters to replacements. `translate()` applies that table. This is the most efficient way to do multiple character substitutions in one pass.

```
# Replace vowels with numbers
table = str.maketrans("aeiou", "12345")
s = "hello world"
print(s.translate(table)) # h2ll4 w4rld

# Three-argument form: from, to, delete
# Arg 1: characters to replace
# Arg 2: what to replace with (same length!)
# Arg 3: characters to DELETE
table2 = str.maketrans("aeiou", "AEIOU", " ")
print("hello world".translate(table2)) # hEll0w0rld (space deleted)

# Delete characters without replacement
delete_table = str.maketrans("", "", "aeiou")
print("hello world".translate(delete_table)) # hll wrld

# Caesar cipher (shift by 1)
import string
alpha = string.ascii_lowercase
shifted = alpha[1:] + alpha[0] # bcdef...za
cipher = str.maketrans(alpha, shifted)
print("hello".translate(cipher)) # ifmmp
```

Tip: `maketrans()` is a static method on the `str` class. Always call it as `str.maketrans()`, not on a string instance.

String Formatting Operator (% style)

The `%` operator for strings is the old-style formatting. Still appears in exams. It uses format specifiers like `%s` (string), `%d` (integer), `%f` (float).

```
name = "Raj"
marks = 85
avg = 85.5

# %s = string, %d = integer, %f = float
print("Name: %s, Marks: %d" % (name, marks))
```



```

# Name: Raj, Marks: 85

# %.2f = float with 2 decimal places
print("Average: %.2f" % avg)

# Average: 85.50

# %10s = right-align in 10 chars
# %-10s = left-align in 10 chars
print("|%10s|" % "hi") # | hi|
print("|%-10s|" % "hi") # |hi |

# Compare with f-string (modern, preferred)
print(f"Name: {name}, Marks: {marks}") # same result

```

All 3 styles exist: % operator (old), .format() (medium), f-strings (modern). f-strings are preferred but % operator still appears in lab questions.

finditer() — Find All with Position Info

Unlike findall() which returns a list of strings, re.finditer() returns an **iterator of match objects**. Each match object has .group(), .start(), and .end() — useful when you need positions.

```

import re

text = "Call 9876543210 or 8765432109 for support"

# findall returns list of strings
print(re.findall(r'\d{10}', text))
# ['9876543210', '8765432109']

# finditer returns match objects with positions
for match in re.finditer(r'\d{10}', text):
    print(f"Found: {match.group()}")
    print(f" Start: {match.start()}, End: {match.end()}")
    print(f" Span: {match.span()}")

# Found: 9876543210
# Start: 5, End: 15, Span: (5, 15)
# Found: 8765432109
# Start: 19, End: 29, Span: (19, 29)

```

re function	Returns
re.match(p,s)	Match object or None — checks START only
re.search(p,s)	Match object or None — scans anywhere
re.findall(p,s)	List of strings — all matches
re.finditer(p,s)	Iterator of match objects — includes positions
re.sub(p,r,s)	New string with matches replaced by r

Topic 4 — Tuples: Packing, Unpacking & Operations

Tuples — Packing, unpacking, singleton rule, copy operation, comparison

Packing and Unpacking

Packing is assigning multiple values to a single tuple. **Unpacking** is extracting tuple elements into individual variables. The number of variables must match the number of elements — or use `*` to capture the rest.

```
# PACKING — multiple values become one tuple
coords = 10, 20, 30 # parentheses optional when packing
print(coords) # (10, 20, 30)
print(type(coords)) #

# UNPACKING — one tuple becomes multiple variables
x, y, z = coords
print(x, y, z) # 10 20 30

# Extended unpacking with * (captures rest)
first, *rest = (1, 2, 3, 4, 5)
print(first) # 1
print(rest) # [2, 3, 4, 5] — note: list, not tuple

*start, last = (1, 2, 3, 4, 5)
print(start) # [1, 2, 3, 4]
print(last) # 5

a, *mid, b = (1, 2, 3, 4, 5)
print(a, mid, b) # 1 [2, 3, 4] 5

# Swap using unpacking (no temp variable)
a, b = 10, 20
a, b = b, a
print(a, b) # 20 10
```

Singleton Tuple — The Trailing Comma Rule

```
# WRONG — this is NOT a tuple, just an int in parentheses
x = (5)
print(type(x)) #

# CORRECT — trailing comma makes it a tuple
x = (5,)
print(type(x)) #

# Also valid without parentheses
x = 5,
print(type(x)) #

# Empty tuple
```

```
empty = ()  
print(type(empty)) #
```

Watch out: (5) is integer 5. (5,) is a tuple. This is one of the most common Python mistakes.

Copy Operation on Tuples

Because tuples are **immutable**, copying works differently from lists. A simple assignment creates a reference to the same tuple object — but since tuples can't be modified, this is usually safe. Use `tuple()` or slicing for an explicit copy.

```
t1 = (1, 2, 3)  
  
# Assignment – both point to same object  
t2 = t1  
print(t1 is t2) # True (same object)  
  
# Explicit copy using tuple() constructor  
t3 = tuple(t1)  
print(t1 is t3) # False (new object, same values)  
  
# Slice copy  
t4 = t1[:]  
print(t1 is t4) # False  
  
# Since tuples are immutable, this is safe:  
t2 = t1 # "modifying" t2 actually creates a new tuple  
t2 = t2 + (4,) # t2 is now (1,2,3,4) but t1 unchanged  
print(t1) # (1, 2, 3)  
print(t2) # (1, 2, 3, 4)
```

Tuple Comparison

Tuples are compared **lexicographically** — element by element from left to right. The first unequal element determines the result.

```
print((1, 2, 3) == (1, 2, 3)) # True  
print((1, 2, 3) < (1, 2, 4)) # True (3 < 4 at index 2)  
print((1, 3) > (1, 2, 9)) # True (3 > 2 at index 1 – stops here)  
print((1,) < (1, 0)) # True (shorter tuple is smaller when prefix matches)  
  
# Tuples can be sorted  
students = [("Raj", 85), ("Priya", 90), ("Sam", 75)]  
students.sort() # sorts by first element (name)  
print(students)  
# [('Priya', 90), ('Raj', 85), ('Sam', 75)]  
  
students.sort(key=lambda x: x[1], reverse=True) # sort by marks desc  
print(students)
```

```
# [('Priya', 90), ('Raj', 85), ('Sam', 75)]
```

Topic 5 — Sets: frozenset, Set Comprehension, all() & any()

Sets — frozenset, Set Comprehension, all() and any()

frozenset — Immutable Set

A **frozenset** is an immutable version of a set. Once created, elements cannot be added or removed. Because it is hashable (immutable), it can be used as a **dictionary key** or as an element inside another set — regular sets cannot.

```
# Create frozenset
fs = frozenset([1, 2, 3, 4, 5])
print(fs) # frozenset({1, 2, 3, 4, 5})
print(type(fs)) #

# Immutable — these raise AttributeError:
# fs.add(6) -> AttributeError
# fs.remove(1) -> AttributeError

# But set OPERATIONS that return a NEW frozenset work fine
fs2 = frozenset([3, 4, 5, 6, 7])
print(fs & fs2) # frozenset({3, 4, 5}) intersection
print(fs | fs2) # frozenset({1,2,3,4,5,6,7}) union

# frozenset as dictionary key (regular set cannot be a key)
d = {frozenset([1,2]): "pair", frozenset([3]): "single"}
print(d[frozenset([1,2])]) # pair

# frozenset inside a set
s = {frozenset([1,2]), frozenset([3,4])}
print(s)
```

Feature	set
Mutable	Yes — add/remove work
frozenset	No — immutable after creation
Usable as dict key	frozenset only
Element of another set	frozenset only
Set operations (,&,-,^)	Both support them

Set Comprehension

Set comprehension is like list comprehension but produces a **set** (unique, unordered). Use curly braces { }.

```
# Set of squares from 1 to 10
squares = {x**2 for x in range(1, 11)}
```

```

print(squares)
# {1, 4, 9, 16, 25, 36, 49, 64, 81, 100} (order may vary)

# With condition – only even squares
even_sq = {x**2 for x in range(1, 11) if x % 2 == 0}
print(even_sq) # {4, 16, 36, 64, 100}

# Remove duplicates from a list using set comprehension
words = ["hello", "world", "hello", "python", "world"]
unique = {w.lower() for w in words}
print(unique) # {'hello', 'world', 'python'}

# Contrast with list comprehension:
list_comp = [x**2 for x in range(5)] # list, allows duplicates
set_comp = {x**2 for x in range(5)} # set, unique values only

```

all() and any() — Testing Multiple Conditions

`all(iterable)` returns True only if **every** element is truthy. `any(iterable)` returns True if **at least one** element is truthy. Both short-circuit — they stop as soon as the result is known.

```

# all() – every element must be truthy
print(all([True, True, True])) # True
print(all([True, False, True])) # False
print(all([])) # True (vacuously true)

# any() – at least one must be truthy
print(any([False, False, True])) # True
print(any([False, False, False])) # False
print(any([])) # False (nothing is true)

# Practical: check all marks pass
marks = [75, 82, 90, 61, 70]
print(all(m >= 50 for m in marks)) # True (everyone passed)
print(any(m >= 90 for m in marks)) # True (someone scored 90+)

# Check if any string starts with 'P'
names = ["Raj", "Priya", "Sam"]
print(any(n.startswith("P") for n in names)) # True
print(all(n.startswith("P") for n in names)) # False

```

Short-circuit: `all()` stops at first False. `any()` stops at first True. This matters for performance with large iterables — Python won't check the rest once the answer is determined.

Topic 6 — Dictionary Comprehension & Advanced Dict Operations

Dictionaries — Comprehensions, merging, advanced iteration patterns

Dictionary Comprehension

Creates a dictionary in one line using the syntax `{ key_expr : value_expr for item in iterable if condition }`.

```
# Basic: number -> square
squares = {x: x**2 for x in range(1, 6)}
print(squares)
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# With condition – only even keys
even_sq = {x: x**2 for x in range(1, 11) if x % 2 == 0}
print(even_sq)
# {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}

# Invert a dictionary (swap keys and values)
original = {"a": 1, "b": 2, "c": 3}
inverted = {v: k for k, v in original.items()}
print(inverted) # {1: 'a', 2: 'b', 3: 'c'}

# Word length dictionary
words = ["hello", "world", "python"]
lengths = {w: len(w) for w in words}
print(lengths) # {'hello': 5, 'world': 5, 'python': 6}

# Filter dictionary – keep only passing marks
marks = {"Raj":85, "Priya":42, "Sam":78, "Ali":35}
passed = {k: v for k, v in marks.items() if v >= 50}
print(passed) # {'Raj': 85, 'Sam': 78}
```

Merging Dictionaries

Python provides multiple ways to merge dictionaries:

```
d1 = {"a": 1, "b": 2}
d2 = {"b": 20, "c": 3} # note: "b" exists in both

# Method 1: update() – modifies d1 in place
d1.update(d2)
print(d1) # {"a": 1, "b": 20, "c": 3} (d2 wins on conflict)

# Method 2: {**d1, **d2} – creates new merged dict (Python 3.5+)
d1 = {"a": 1, "b": 2}
merged = {**d1, **d2}
print(merged) # {'a': 1, 'b': 20, 'c': 3}
```

```
# Method 3: | operator (Python 3.9+)
merged2 = d1 | d2
print(merged2) # {'a': 1, 'b': 20, 'c': 3}

# |= in-place merge (Python 3.9+)
d1 |= d2
print(d1) # {'a': 1, 'b': 20, 'c': 3}
```

Iterating Over Dictionaries

```
student = {"name": "Raj", "age": 20, "marks": 85}

# Iterate over keys (default)
for key in student:
    print(key) # name, age, marks

# Iterate over values
for value in student.values():
    print(value) # Raj, 20, 85

# Iterate over key-value pairs
for key, value in student.items():
    print(f"{key}: {value}")

# Enumerate for index too
for i, (k, v) in enumerate(student.items()):
    print(f"{i}. {k} = {v}")

# Dictionary as counter (frequency map)
sentence = "the cat sat on the mat"
freq = {}
for word in sentence.split():
    freq[word] = freq.get(word, 0) + 1
print(freq)

# {'the': 2, 'cat': 1, 'sat': 1, 'on': 1, 'mat': 1}
```


Topic 7 — Modules: `dir()`, Namespaces & `fromimport`

Modules — dir(), Namespaces, import styles, creating your own module

Import Styles — All Variations

Import style	How to call it
<code>import math</code>	<code>math.sqrt(16)</code> — must use module name
<code>import math as m</code>	<code>m.sqrt(16)</code> — alias
<code>from math import sqrt</code>	<code>sqrt(16)</code> — direct, no prefix
<code>from math import sqrt, pi</code>	<code>sqrt(16), pi</code> — multiple imports
<code>from math import *</code>	<code>sqrt(16)</code> — imports everything (avoid!)

Creating Your Own Module

Any Python file is a module. Save functions in a `.py` file, then import it by filename (without `.py`). The file must be in the same folder or in `sys.path`.

[illegible]

Remember: The `if __name__ == '__main__':` block is important. It lets a file work both as a runnable script AND as an importable module. When imported, `__name__` is the module name, not `'__main__'`.

dir() — Inspect a Module

`dir()` returns a sorted list of names in a module, object, or the current scope. Very useful for discovering what a module offers.

```
import math
print(dir(math))

# ['__doc__', '__loader__', '__name__', '__package__', '__spec__',
# 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh',
# 'ceil', 'comb', 'copysign', 'cos', 'cosh', 'degrees', 'dist',
# 'e', 'erf', ... 'sqrt', 'tan', 'tanh', 'tau', 'trunc']

# dir() on an object shows its methods
s = "hello"
print(dir(s)) # shows all string methods

# dir() with no argument shows current scope
x = 10
def my_func(): pass
print(dir()) # ['my_func', 'x', ...]

# __doc__ shows module docstring
import os
print(os.__doc__[:100]) # first 100 chars of os module docs
```

Namespaces — Why Names Don't Clash

A **namespace** is a mapping from names to objects. Python has four nested namespaces: **LEGB** — Local, Enclosing, Global, Built-in. Python searches them in this order.

```
# LEGB Rule demonstration
x = "global" # Global namespace

def outer():
    x = "enclosing" # Enclosing namespace

    def inner():
        x = "local" # Local namespace
        print(x) # local (finds Local first)

    inner()
    print(x) # enclosing

outer()
print(x) # global

# Built-in namespace — len, print, etc.
# Python checks: Local -> Enclosing -> Global -> Built-in

# Module namespaces — each module has its own namespace
import math
```

```
import cmath  
  
# math.sqrt and cmath.sqrt are DIFFERENT functions  
# but no naming clash because they're in different namespaces
```

Tip: When you do 'from math import sqrt', you copy the name 'sqrt' into the current (global) namespace. If you later do 'from cmath import sqrt', it **OVERWRITES** the previous sqrt. This is why 'import *' is risky.

Topic 8 — Variable Scope, Lifetime & Nested Functions

Scope & Lifetime — Variable scope, global, nonlocal, closures, LEGB rule

Variable Lifetime vs Scope

Concept	Meaning
Scope	Where in the code a variable can be accessed (spatial)
Lifetime	How long the variable exists in memory (temporal)
Local var	Exists only while function runs — destroyed when function returns
Global var	Exists for the entire program duration

```
# Local variable — dies when function returns
def my_func():
    local_x = 100 # created when function called
    print(local_x) # 100
my_func()
# print(local_x) -> NameError: not defined anymore

# Global variable — lives entire program
counter = 0

def increment():
    global counter # MUST declare global to modify it
    counter += 1

increment()
increment()
print(counter) # 2

# Without global keyword — creates a LOCAL variable instead!
def broken():
    counter += 1 # UnboundLocalError — Python thinks it's local
    # because of assignment, but reads it first
```

Watch out: If a function assigns to a variable name AND reads it without 'global', Python treats it as local for the ENTIRE function — even lines before the assignment. This causes UnboundLocalError.

nonlocal — Modify Enclosing Scope

In nested functions, `nonlocal` lets an inner function modify a variable in the enclosing (but not global) scope. It's the nested-function equivalent of `global`.

```
def make_counter():
    count = 0 # enclosing scope
```

```
def increment():
    nonlocal count # access enclosing scope variable
    count += 1
    return count

return increment

c = make_counter()
print(c()) # 1
print(c()) # 2
print(c()) # 3
# Each call remembers the previous count — this is a CLOSURE
```

Closure — Function That Remembers Its Environment

A **closure** is an inner function that captures variables from its enclosing scope, even after the outer function has returned. The captured variables are stored in the function's `__closure__` attribute.

```
# Closure: inner function remembers 'multiplier' from outer scope
def make_multiplier(multiplier):
    def multiply(x):
        return x * multiplier # captures 'multiplier' from outer scope
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)

print(double(5)) # 10
print(triple(5)) # 15

# Both are closures — each remembers its own multiplier
print(double.__closure__[0].cell_contents) # 2
print(triple.__closure__[0].cell_contents) # 3
```

Tip: Closures are the foundation of decorators in Python. They are also used to create factory functions — functions that generate other functions.

Quick Reference Card — All Gap Topics

All gap topics summarised — syntax, purpose and key points at a glance

Topic	Key Point / Syntax
Lambda Functions	lambda args: expr square = lambda x: x**2
filter()	list(filter(lambda x: x>0, nums)) — returns subset
map()	list(map(lambda x: x**2, nums)) — transforms each
reduce()	from functools import reduce; reduce(lambda a,b: a+b, lst)
*args	def f(*args): — collects extra positional args as tuple
kwargs	def f(kw): — collects extra keyword args as dict
Docstring	First string in function body; access via func.__doc__
s.title()	'the fox'.title() -> 'The Fox' (every word)
s.capitalize()	'hello WORLD'.capitalize() -> 'Hello world' (first only)
s.swapcase()	'Hello'.swapcase() -> 'hELLO'
s.startswith(t)	s.startswith(('pdf','docx')) — tuple of prefixes
s.endswith(t)	s.endswith('.pdf', 'txt')) — tuple of suffixes
str.maketrans(f,t,d)	Creates char translation table; apply with s.translate(tbl)
% formatting	'Name: %s, Age: %d' % (name, age)
re.finditer()	Returns match objects with .group(), .start(), .end()
Tuple packing	t = 1, 2, 3 — parentheses optional
Tuple unpacking	a, b, c = t OR first, *rest = t
Singleton tuple	(5,) not (5) — trailing comma required
frozenset	fs = frozenset([1,2,3]) — immutable, hashable set
Set comprehension	{x**2 for x in range(5) if x%2==0}
all()	all(m>=50 for m in marks) — True if ALL truthy
any()	any(m>=90 for m in marks) — True if ANY truthy
Dict comprehension	{k:v for k,v in d.items() if v>50}
Merge dicts	{**d1, **d2} or d1 d2 (Python 3.9+)
dir()	dir(math) — list of all names in module/object
__name__ == '__main__'	Runs only when file executed directly, not when imported
global keyword	Inside function: global x — allows modifying global x
nonlocal keyword	Inside nested function: modifies enclosing scope variable
Closure	Inner function capturing enclosing scope variables
LEGB Rule	Python name lookup: Local -> Enclosing -> Global -> Built-in